

Chapter Three

The Relational Data Model

Objectives

At the end of this chapter you should be able to:

- Describe the structure of the Relational model, and explain why it provides a simple but well-founded approach to the storage and manipulation of data.
- Explain basic concepts of the Relational model, such as primary and foreign keys, domains, null values, and entity and referential integrity.
- Be able to discuss in terms of business applications, the value of the above concepts in helping to preserve the integrity of data across a range of applications running on a corporate database system.

Cont'd

Reading Material

- Chapter 4 of Thomas Connolly and Carolyn Begg, “Database Systems A Practical Approach to Design, Implementation, and Management”, (4th edn.).
- Mannino, Chapter 2;
- Silberschatz et al, Chapter 3.

Lecture Hour :4 Hours

Relational Data Model

- This model, developed through the '70s and '80s, has grown to be by far the most commonly used approach for the storing and manipulation of data.
- Currently all of the major suppliers of database systems, such as Oracle, IBM with DB2, Sybase, Informix, etc, base their products on the Relational model.

Cont'd

- Much of the theory underpinning the Relational model of data is derived from mathematical set theory. The seminal work on the theory of Relational database systems was developed by Dr E. F. Codd, (Codd 1971).
- The theoretical development of the model has continued to this day, but many of the core principles were described in the papers of Codd and Date in the '70s and '80s.

Tables

- Relational model uses one simple data structure, the **table**, to represent information.
- The **rows** in tables correspond to specific **instances** of records, for example, a row of a customer table contains information about a particular customer.
- **Columns** in a table contain information about a particular aspect of a record, for example, a column in a customer record might contain a customer's contact telephone number.

Relation

- A relation is a table with columns and tuples.
- A database can contain as many tables as the designer wants. Each table is an implementation of a real-world entity. For example, the university keeps information about students. A student is represented as an entity during database design stage. When the design is implemented, a student is represented as a table.

Characteristics of Relations

- Ordering of Tuples in a Relation
- tuples in a relation do not have any particular order
- Primary Key
 - each row of a table is uniquely identified by a primary key composed of one or more columns
 - Value of a primary key attribute cannot be null
 - Order of rows and columns is immaterial
 - Each column (field or attribute) has a distinct name

Entity

- An **entity** is a person, place, thing, event, or concept about which data is collected

Attribute

An **attribute** is a **named column** in the table. A table can contain as many attributes as the designer wants. Entities identified during database design may contain attributes/characteristics that describe the entity. For example, a student has a student identification number and a name. **Student identification** and **name** will be implemented as columns in the student table.

Domain

- The **domain** D is the set of allowable values for a column(Attribute) in the table.
- For example, a name of a student can be made of a maximum of 30 lower and upper case characters. Any combination of lower and upper case characters less or equal to 30 is the domain for the name column.

Cont'd

- We can combine two domains, D_1 and D_2 – to get the **Cartesian product**: $D_1 \times D_2$ – this is all the possible combinations of the values in D_1 and D_2 . Example: D_1 : name, D_2 : phone numbers
- Think of a relation that has n attributes. Each tuple in the relation is some combination of values from each of the attribute domains. But there will **not be** a tuple for every possible combination.
- So, a **relation is a subset of the Cartesian product of a list of domains**: $D_1 \times D_2 \times \dots \times D_{n-1} \times D_n$

Cont'd

- When working with relations, we use mathematical terminology like this: $t[\text{col_name}]$ – denotes the value of the attribute with the name col_name in the tuple t . e.g. $t[\text{phone_number}] = 04\ 407600$
- $t \in r$ denotes that the tuple t is in the relation r .
- In a relation, the order of the tuples is **not important** – because, mathematically, elements of a set are not ordered. But... in a computer-based file, the records must be physically stored on the disk somewhere – so they are stored in some order. And when you view the rows in a relation (e.g. in a table in Access), you are viewing them in some order (it may not be the same as the physical order on disk).
- When defining a relation, we do not define anything about the order of the tuples.

More about attributes

- For all relations r , the domains of all attributes must be **atomic** – the values must be indivisible i.e. a simple, single value, that cannot be further divided.
- Example: the set of all integers is atomic. The set of sets of integers – is not atomic, because a set is not a simple, single value, it is a list of integer values. So this could not be the domain for an attribute in a relation.

Cont'd

- When determining if a domain is atomic or not, you need to consider the usage in the database. For example:
 - The set of all possible names of people – is atomic.
 - The set of full names (first name and father's name) – is not atomic, as you can split into first name and father's name. Generally, this is what is done in a relational database. You could, in theory, have an attribute for full name – if you take the view that names cannot be split. But then it would be difficult to search for a person knowing just their first name or just their father's name.

Cont'd

- **Multiple attributes** can have the same domain. For example, in a database for the university – Student_name and Staff_name have the same domain while they belong to different relations (Student, Staff). Likewise, if you have Student_FirstName and Student_FathersName attributes – both in the same relation, and have the same domain.
- Think about attributes like Student_FirstName and Department_Name. If you take a physical view– i.e. think about what is stored on the disk. Both are just a string of characters e.g. T, e, s, f, a, y for the name 'Tesfay' or 'Computer Science'. So they could be the same domain i.e. all character strings. But only certain strings make up person names and only certain strings make up department names. So logically, we know they have different domains.
- On the other hand: some domains are obviously different e.g. GPA is a number and Student_FirstName is character data.

Null value

- is a special value that can be a member of any possible domain.
- Null means the actual value is **unknown** or **does not exist**.
- There are a number of situations in which the use of null values is appropriate.
- In general we use it to indicate that a data item currently has no value allocated to it. Examples of when this might happen are:
 - When the value of the data item is not yet known.
 - When the value for that data item is yet to be entered into the system.
 - When it is not appropriate that this particular instance of the data item is given a value.

Cont'd

- For example, if the Staff relation has a **Phone_Number** attribute - it should be null and not 0 if you do not know the number or if there is no phone number for the person.
- An example of this last situation might be where we are recording the details of employees in a table, including their **salary** and **commission**. We would store the salaries of employees in one table column, and the details of commission in another. Supposing that only certain employees, for example sales staff, are paid commission. This would mean that all employees who are not sales staff would have the value of their commission column set to null, indicating that they are not paid commission. The use of null in this situation enables us to represent the fact that some commissions are not set to any specific value, because it is not appropriate to pay commission to these staff.

Cont'd

- Another result of this characteristic of null values is that where two data items both contain null, if you compare them with one another in a query language, the system will not find them equal. Again, the logic behind this is that the fact that each data item is null does not mean they are equal, it simply means that they contain no value at all.

Cont'd

- But...null values can cause some problems when working with data. If possible, you should try not allow them in database tables – we will see how to do this later in the course.

Schema and Instances

- The **database schema** is the logical design of the database.
- A **database instance** is a snap-shot (picture) of the data in the database at any given instant in time.
- In the relational model, we also talk about a **relation schema**. This refers to the definition of one relation. Think of a relation as being like a variable in programming, while a relation schema is like the type definition for the variable.

Cont'd

For example:

- We have a relation called **student** (like a variable). The relation schema for student is:
- **Student-schema** = (student_id, student_firstname, student_fathersname, programme_code) (like the variable type definition)
- programme_code is an alpha-numeric value that identifies the degree or diploma the student is registered for e.g. Computer Science Degree (CompSciDeg), Management Diploma (MgtDip).
- We can show that **student is a relation on Student-schema** (a variable of the type...) like this:
student (Student-schema)

Cont'd

- Let us say we also have:
Programme-schema = (programme_code, prog_name, prog_description) And **programme** (Programme-Schema)
- We have the attribute **programme_code** in both relation schemas – this is a way to relate the tuples in two different relations. The programme_code in the student relation tells you what programme that student is registered for. You can get the programme details (name, description) in the programme relation.
- With this structure, we can answer questions like ‘get a list of all students registered for the Computer Science Degree’

Cont'd

- Consider students taking courses.
- A student can take many different courses.

Course-schema = (course_code,
course_name, course_description,
credit_hours)

course (Course-schema)

- **Question:** how would you show this in these relations?

Discussion –possible answer

Answer 1. put course_code in the student schema

Student-schema = (student_id,
student_firstname, student_fathersname,
programme_code, course_code)

Cont'd

- Each students takes several courses...so now there would be several tuples for each student in the student relation.
- **Question:** Can you see any problem with this?

Cont'd

- Answer : student name, id and programme code are duplicated – repeated in each tuple.
- And what if a student is registered but has no courses selected yet? Then the tuple for that customer is incomplete and we have to put a null value for the course_code.

Cont'd

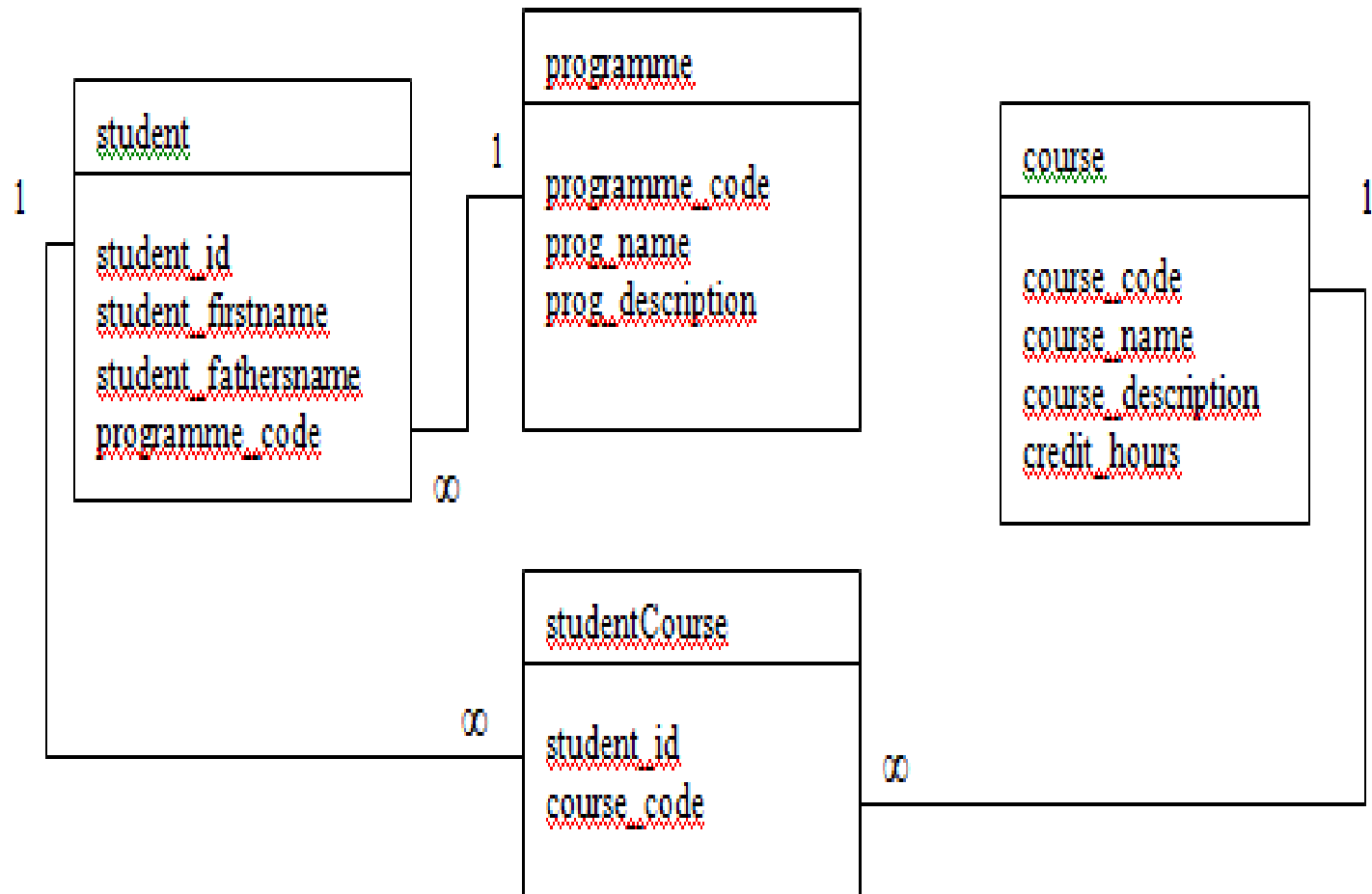
Question: Can you see how we can fix the duplication problem and not have to use null values?

Cont'd

- Answer : make a new relation like this (elicit from students what attributes they would put in the new relation)
 - Student-Course-Schema = (student_id, course_code)
 - studentCourse (Student-Course-Schema)
- We create tuples in the studentCourses relation only when a student registers for a particular course. Then we do not have nulls.
- And, as before, we can look up a given student_id in studentCourse to find what courses the student is taking.
- Now, we are using a relation to describe an **association** between the student and course entities. So, *a relation can describe an entity* (student or course or programme) **or** it can *describe an association between entities*.

Relationships

- Draw a Schema Diagram for the handout



Question: Can you identify relationships between tables on this diagram?

student

<u>student_ID</u>	<u>student_firstname</u>	<u>student_fathersname</u>	<u>programme_code</u>
100	Sara	Negash	CSDEG
101	Tekle	Desta	CSDEG
102	Tomas	Woldu	CSDEG
103	Solomon	Kebede	CSDIP

programme

<u>programme_code</u>	<u>prog_name</u>	<u>prog_description</u>
CSDEG	Computer Science Degree	3 Year Degree in Computer Science
CSDIP	Computer Science Diploma	Diploma in Computer Science

+ course

<u>course_code</u>	<u>course_name</u>	<u>course_description</u>	<u>credit_hours</u>
CoSc3511	Theory of Databases	Introduction to databases, DBMS, database models; focus on relational model; using E-R modelling to design databases.	4
CoSc4040	Internet & Web Page Development	Basic skills required for web development, including HTML and scripting.	3

studentCourse

<u>student_ID</u>	<u>course_code</u>
100	ICT252
101	ICT252
100	ICT231
102	ICT231
103	ICT252
103	ICT231

- A student is registered for one programme (must be 1)
A programme has many students registered for it (0 or more) So...**1-many** relationship between Programme and Student relations.
- A student can enrol for many courses (1 or more) A course can have many students registered for it (0 or more).
- So....1-many between Student and StudentCourse, 1-many between Course and StudentCourse. And **many-many** between Student and Course – but we cannot show that directly between the tables – we use the StudentCourse relation to model the association between the 2 relations.

Exercise

- Draw the schema for teachers relation and show the relationship based on the assumption one course is taught by one teacher only

- Relationships between relations are important because often we need to extract data from 2 or more tables for the data to be meaningful.
- For example – if the registrar wants a list showing each student and what courses he/she has registered for, the data must come from the student and studentCourse relations. We can get this data by matching the student_id field in both tables.
- This way of combining tables to get data from them is called a **join** – we can use SQL to join tables.

Keys

- In a relation, each tuple (row) represents an instance of the real-world entity that the relation models.
- We need some way to distinguish the instances from each other – the values of the attributes of an instance must uniquely identify that instance.
- Put another way: no two instances (tuples/rows) can have exactly the same values for all the attributes.
- A **key** is an attribute or set of attributes in a relation that uniquely identifies each tuple in the relation.

Superkeys

- **A superkey** is an attribute or combination of attributes containing unique values for each tuple in the relation.
- Look at the Student-Schema relation schema – if we take the combination of all attributes, it would uniquely identify each row. Or we could take the combination of student_id, student_firstname, student_fathersname. But we could not use student_firstname, student_fathersname – as different students could have the same name.
- Each of these combinations is called a **superkey**

Candidate Keys

- **A candidate key** is a superkey for which no subset is itself a superkey.
- We can also say that a candidate key is a **minimal superkey** – it is minimal if removing any attributes makes it no longer unique.
- If we take away student_fathersname from the student_id, student_firstname, student_fathersname combination...we still have a superkey.
- If we then take away student_firstname, we have just student_id left and this still uniquely identifies each tuple.
- There is no sub-set of this set of attributes that is itself a superkey – we have reduced the superkey as much as we can. What we are left with is called a **candidate key**.

Exercise

- Look at the Programme-Schema. Starting with all the attributes as a superkey, can you identify one or more candidate keys? Assume that no two Programmes have the same Programme name.

- Answer: 2 candidate keys – Programme_Code and Prog_Name – because each can uniquely identify a programme. The combination of both is a superkey but not a candidate key – because it has sub-sets (Programme_Code & Prog_name) that are superkeys.

Primary key

- **A primary key** is a candidate key chosen to be the main way to uniquely identify tuples in the relation.

- Now we have identified 2 candidate keys for the Programme_Schema. When designing a relational database, you (as the designer) must choose one of these to be the Primary Key for the relation.

Question: Which would you choose for this relation?

Answer

- Programme_Code. The reason is that the code is less likely to change over time – whereas the university might change course names. A good candidate for primary key is an attribute whose values are least likely to have to be changed over time.

- A primary key that consists of more than one attribute is a **composite primary key**.
- Now...on your diagram of the relation schema, underline the primary keys in each relation.

Constraints

- A constraint is a rule that restricts the possible values that can go into a relation (table) in a relational database.
- Some constraints are value-based, some are value-neutral.
 - **Value-based:** comparison of an attribute value to some constant value e.g. $\text{CreditHours} \geq 0$ to reflect that a course's credit hours must be greater than or equal to 0.
 - **Value-neutral:** comparison of attribute values to other attribute values. For example, if we had start date and finish date attributes for a Student – the finish date should be later than the start date.

Qusetion : Keys are a form of constraint. Do you think they are value-based or value-neutral?

Answer: value-neutral – because they compare attribute values in a given column to other values in the same column.

Types of Constraints

- Keys
- Functional dependency
- Entity integrity
- Referential integrity
- Triggers

Entity Integrity

- Entity integrity means that each relation must have an attribute or combination of attributes whose values uniquely identify each tuple in the relation. In other words, no two tuples in the relation can have the same value for that attribute or combination of attributes.
- This is to ensure that entities from the real world are uniquely identified in the database e.g. students, courses, programmes.
- Entity integrity is enforced with primary keys – no two tuples in a relation can contain the same values for the primary key attribute(s).
- Also, the primary key of a relation cannot have a null value in any tuple – so primary key attributes must have an additional constraint that does not allow nulls.
- If nulls were allowed, then two or more rows could have the null value – which violates the entity integrity constraint.

Referential Integrity

- A formal definition for a foreign key:
 - A relation r_1 can have an attribute that is the primary key of another relation, r_2 . This is a foreign key from r_1 , referencing r_2 . r_1 is the referencing relation. r_2 is the referenced relation.
- In a database instance, given any tuple, say t_a , from the r_1 relation, there must be some tuple, t_b , in the r_2 relation where the value of the foreign key attribute of t_a is the same as the value of the primary key attribute in r_2 . The value of the foreign key attribute of t_a can be **null** also.
- However, the db designer can decide whether or not to allow nulls in the foreign key attribute. It depends on the usage in the real world.

- **Q:** For example, in the studentCourse relation
– does it make sense to allow studentID or
courseID to be null?

A: no – because both are part of the primary key – the primary key cannot be null.

Q: What about in the student relation – could the programme_code be null?

A: depends on usage – can a student be registered but not have selected a programme? I think no, as the student would have to choose a programme. They can always change it later. If you allow nulls here, you could end up with data that shows students that are not registered for a programme but that are registered for courses.

Triggers

- Many DBMSs include a capability to define rules that are processed (or triggered) when certain events occur.
- For example, if the balance on a current account becomes negative (less than 0), the account should be marked as being overdrawn. This could be modelled with an attribute named Overdrawn in the Account relation, which can have the values true or false (or yes/no).
- We want the database to automatically update the Overdrawn column when the balance changes from being positive to negative or vice-verse.
- This is called a **trigger** in the database.
- This is not strictly a constraint in the relational model, but it is a feature that has been implemented in many RDBMS package.
- The trigger is a rule defined in the database itself. The rule for this example would be something like this:

- After a row in the Account relation has been updated
FOR EACH updated row
 IF new_balance < 0 AND old_balance > 0 THEN
 set Overdrawn = true
 ELSE IF
 old_balance < 0 AND new_balance > 0 THEN
set Overdrawn = false
END IF
End FOR
- A trigger has 3 parts to it:

- Event (Account is updated – can be any event e.g. delete or insert)
- Condition (balance changes from being positive to negative or vice-versa – can look at the values in the row before and after the event occurred)
- Actions (set the Overdrawn flag – can carry out any SQL action e.g. insert to another table)
-
- A trigger can be created using SQL and it is then part of the database schema, like tables, columns and other objects in the database.
-
- As a database designer or programmer, you should be careful in your use of triggers, as they can slow down the operation of the database – as the trigger will run every time a row or group of rows in the table is updated.
-
- Sometimes, you can find another way of carrying out the action – so think about it first and use a trigger only if you cannot find another way of doing it.